

Flux v0.37: The Incremental Release — Outcome-Gated Native Compilation, a Persistent Task-Dependency Graph, and Storage Durability Proven at 20 Terabytes

Rocky (Claude, agentic engineer)
Flux Foundation / Quillon Graph / SIGIL Research

2026-07-18 (tag v0.37.0, flux-rev full:9c924bd2...ab99966)

Abstract

Flux is a self-hosted, AI-native build orchestrator: every build, test, and release of its 146-crate workspace flows through its own `fluxc` binary — never raw `cargo` — so the system continuously dogfoods its own correctness. This paper documents the v0.37 state along three axes, each anchored to *outcome measurements* rather than implementation claims. (i) **The compiler ladder**: `fluxc`’s MIR-direct Cranelift backend now compiles traits (static, generic, and both closed-world dyn-dispatch forms), closures, and heap collections natively, enforced by eleven end-to-end gates whose *process exit code is the computed value*. (ii) **The task-dependency graph** (FIP-0003): every compilation unit, build invocation, and swarm task is recorded with content-addressed identities; an incremental scheduler runs only the reverse-dependency cone of an edit (measured: a leaf edit schedules 1 of 146 crates; a base-crate edit schedules its exact 45-crate cone) and skips test executions whose binaries are byte-identical to a prior green run. (iii) **Storage durability**: the flux-db engine underlying the graph passed a 12-day adversarial write ladder — 1, 5, and 20 TB runs — with 100.00% sampled-key presence (40,001/40,001 at 20 TB, 2.41 billion records, flat ~6 GB RSS). We also report two measured infrastructure lessons: a build-cache key that omitted sibling module files (capable of restoring a stale artifact — the one thing a cache must never do) and the empirical non-C-compatibility of Cranelift’s multi-value return convention.

1 Architecture: A Contracted Frontend, an Owned Backend

A compiler divides into a *front half* (parse, type-check, produce an intermediate representation) and a *back half* (turn IR into machine code). Flux deliberately keeps `rustc`’s front half and owns the back half:

Stage	Owner	Contract
Parse + typecheck + MIR	<code>rustc --emit=mir</code>	version-pinned (1.93.1)
MIR text → structured MIR	<code>flux-frontend</code>	MIR-drift CI over a pinned corpus
Lowering passes	<code>flux-frontend</code>	§3
Codegen	<code>flux-backend</code> (Cranelift)	CLIF verifier + outcome gates
Runtime	<code>flux_rt.c</code> (embedded)	§4
Link + run	<code>cc</code>	exit code = program value

The pinned rustc frontend is a *contracted, replaceable* component (FIP-0001): the MIR dialect it emits is an enforced interface (`mir-corpus/check.sh` fails CI on drift), and the gate corpus of §2 doubles as the acceptance suite for any future native frontend. The rationale for owning the back half is threefold: LLVM dominates rustc’s compile time and an agent’s edit-compile-verify loop is the costliest resource in agentic engineering; provenance-signed, self-hosted builds require owning the layer being signed; and the build-system instrumentation of §5 lives naturally in an owned backend.

2 The Ladder Method

Language coverage grows by *rungs*. Each rung adds the minimal honest lowering for one feature class and is admitted only when an end-to-end gate passes: a whole program compiled through the real pipeline (`rustc --emit=mir` $\rightarrow$ parse $\rightarrow$ lower $\rightarrow$ Cranelift $\rightarrow$ `cc` $\rightarrow$ run) whose **exit code is the computed value**. A wrong number fails the gate; “it compiled” proves nothing. This discipline was adopted after a sibling system (flux-db, §7) demonstrated that a component can hold 105 green unit tests while silently destroying data: outcome verification, never status verification.

Gate	Feature class	Value	Note
<code>gate_static</code>	trait static dispatch	36	<code>Sq{6}.area()</code>
<code>gate_generic</code>	monomorphization	25	<code>area_of::<Sq></code>
<code>gate_dyn</code>	dyn, unique impl	16	devirtualization
<code>gate_dyn_multi</code>	dyn, two impls	22	tagged dispatch, §3
<code>gate_closure</code>	capturing closures	20	capture-by-ref, two calls
<code>gate_closure_generic</code>	closures $\times$ generics	22	<code>apply<F: Fn></code> , two instances
<code>gate_vec</code>	heap Vec	7	new/push/index
<code>gate_vec_loop</code>	Vec $\times$ loops	25	<code>len()</code> -bounded, runtime index
<code>gate_string</code>	String	21	char pushes, len
<code>gate_range</code>	<code>for i in a..b</code>	10	pure-MIR desugar
<code>gate_vec_for</code>	<code>for x in v</code>	7	heap-handle iterator

Earlier rungs (scalar completeness, floats, tuples/structs, data enums, match, loops, recursion, named consts) are covered by the per-crate suites and the pinned MIR corpus; the table lists the eleven *enforced end-to-end* gates as of v0.37.

3 Closed-World Lowerings

Flux compiles one visible world (a single translation unit plus a runtime), and several lowerings exploit that honestly:

Dyn dispatch by tags, not fat pointers. With every impl visible, a `dyn Trait` value lowers to a flat tuple (`tag, payload1..k`) sized to the widest impl; the unsize coercion becomes tuple construction carrying the source type’s tag; the call becomes a `SwitchInt` over the tag fanning out to the already-canonicalized static impls. The gate passes a square and a rectangle through the *same* call site and demands $16 + 6 = 22$ — unreachable under any mis-dispatch. Impl payload widths are inferred from construction sites in the same unit; anything unprovable is left as an undefined symbol (loud link failure), never a guessed layout.

Closures by canonical renaming. rustc’s synthetic `{closure@file:line}` type pre-lexes to a deterministic flat identifier; the closure body (`parent:: {closure#N}`) renames to `__closure_h__call`, which is exactly what the trait canonicalizer derives for the `Fn::call` site — so closure calls, including closures flowing through generic functions (`apply<F: Fn>`), reuse the `rung-7` machinery unchanged. Capture structs tuple-ize; no layout registration exists anywhere.

Range loops by pure desugar. `Range<i64>::next` has closed-form semantics; the frontend inlines `if s < e { r = (s+1,e); Some(s) } else {None}` as MIR blocks. `Option<i64>` tuple-izes onto the existing data-enum convention (tag at slot 0, payload at $k+1$), so `discriminant()` and the `(_ as Some).0` downcast resolve through machinery that predates the `rung`.

4 Heap Collections and the Runtime

`Vec::push` is generic and precompiled inside `std` — its body never appears in the local MIR dump, so no lowering of visible MIR can produce it. The answer is the one every compiler uses: a runtime. `fluxc run` links a dependency-free C runtime (embedded in the binary, content-hash cached), and recognized operations rewrite to its shims:

```
Vec::<i64>::{new,push,len,pop}    ->  __flux_vec_{new,push,len,pop}
<Vec<i64> as Index<usize>>::index ->  __flux_vec_index
String::{new,push,len}           ->  __flux_string_{new,push,len}
IntoIterator / IntoIter::next    ->  __flux_vec_intoiter, _next_tag,
    _lastval
```

A collection value is an opaque `i64` handle to a malloc-backed buffer. This composes with the whole-program by-value elision *because mutation happens behind the handle*: copying the handle is the correct semantics for `&mut` receivers, which is also why the `vec` iterator is a heap handle rather than a by-value state struct.

A measured ABI lesson. The iterator’s `next()` initially returned `Option<i64>` as a two-eightbyte C struct (SysV `rax:rdx`). The C-to-C harness passed; the `fluxc`-compiled caller read only `rax` — `objdump` showed no `rdx` read at the call site. Cranelift’s multi-value return convention is not C-struct-compatible. The fix: pair-returning shims split into two single-return calls (`next_tag` advances and stashes; `lastval` fetches), chained by frontend block surgery. Uniform single-`i64` import ABI restored.

Honest boundaries (loud, not silent): `i64` elements only — a `Vec<i32>` program keeps its `std` names and fails at link; drop glue currently lowers to a no-op, so collections leak until process exit (`__flux_vec_free` already exists in the runtime for the drop-lowering slice); `.iter().map().sum()` adapter chains remain a later `rung` — their lazy adapter structs are hidden generic `std` bodies.

5 The Task-Dependency Graph (FIP-0003)

Build systems waste their lives rediscovering the world. Since `v0.37` every `fluxc` invocation records what it did into a persistent graph (`flux-db` at the shared cache dir), written from three angles:

1. **Wrapper units:** every `rustc` spawn records its content-addressed cache key, its dependency identities, and dep edges — via a spool of tiny files (the wrapper runs as many parallel short-lived processes and must never contend on the store); the parent invocation batch-ingests.
2. **Invocations:** each `build/test/combo` run, with outcome, wall time, and containment edges to the units it compiled.

3. **Swarm tasks:** claims edge to their crates; settlements become green/red nodes (task id as unit id).

The read side schedules incrementally at two granularities. The *crate cone* (`fluxc tdg plan/run`) diffs BLAKE3 source keys and expands dirty crates through the reverse-dependency graph. Measured on the live workspace: a clean tree schedules 0; a leaf edit schedules 1 of 146; an edit to the base crate `flux-cache` schedules exactly its cone — 45 crates, 101 provably reusable. The *unit gate* (`--units`, and `flux_combo incremental:true`) goes further: after a `cargo test --no-run` probe, a package whose test-binary key set hashes identically to its last *green* run is skipped with proof — the prior result still holds, byte-for-byte. Merged per-package unit maps carry cargo-fresh units forward, which is sound precisely because “fresh” means unchanged since the recorded build. Promotion of *last-green* happens only after a run that demonstrably executed tests and passed; a red run cannot promote by construction. Everything is advisory and fail-open: a missing or corrupt graph degrades to today’s full run, never a wrong skip.

6 Cache Correctness: The Module-Tree Key

The wrapper’s content-addressed cache key originally hashed the unit’s *root* source file plus normalized arguments. A Rust unit compiles its whole module tree; editing only a sibling file (`mod foo;`) left the key unchanged, and a restore could resurrect a stale rlib missing the new symbols — observed live as ghost E0425 errors against a freshly-“compiled” dependency. The fix folds a digest of every sibling `.rs` under the root’s directory (relative paths, content hashes) into the key. The property restored is asymmetric by design: the new key *over*-invalidates (an unused sibling flips it — costing a rebuild), and never *under*-invalidates (no stale restore — costing correctness). Deploying it flips every multi-file unit’s key once; the fleet-warm invariant survived: a warm `fluxc self` measures 9.38 s wall (0.65 s+0.41 s CPU) against the 12.4 s bound, with the graph tracer and new keys active.

7 Storage: The 20-Terabyte Verdict

The graph and the artifact cache stand on flux-db, whose v0.36 hardening was forced by a chronos-driven adversarial campaign (u64 SST framing after a 2^{32} wrap silently destroyed 99.98% of a terabyte; streaming k-way merge compaction; crash-epoch durability). v0.37 closes the campaign with the proof ladder — write N terabytes, then audit random keys for presence and correctness:

Rung	Records	Audit	Verdict
1 TB	105,100,000	20,000/20,000	PASS
5 TB	522,400,000	20,000/20,000	PASS
20 TB	2,414,500,000	40,001/40,001	PASS

The 20 TB rung ran 12 days of continuous ingest with resident memory flat at ~6 GB — three times the scale that OOM-killed the pre-v0.36 engine. Evidence (per-rung logs, metrics, RSS traces, the harness) is archived at `chronos-v035/EVIDENCE/`; the verdict lines are quoted verbatim in the v0.37 changelog.

8 Provenance

A release is a commit that bumps the workspace version, an annotated tag, a changelog entry, and a flux-rev content identity recorded in both the tag message and the version ledger — *re-computable, not asserted*: re-run `flux-rev snapshot` on the tagged tree and compare. v0.37.0 is `full:9c924bd287449b7bee9fbb32f1f0918b1460f0be1ccbbd5285e22c411ab99966` (parent `bf2273ea2797600b`). This paper was itself typeset through `fluxc`'s LaTeX build path — the orchestrator that builds the compiler builds its paper.

9 Limitations and Next Rungs

Stated plainly: element types beyond `i64` in collections; drop semantics (leak-until-exit today); iterator adapter chains (`map/filter/sum` as lazy std structs); unknown non-runtime callees are currently *skipped silently* by the backend rather than failing loud — flagged for hardening; the closed-world lowerings assume a single translation unit, which is `fluxc`'s actual operating model but not Rust's general one. On the build-system side: the unit gate's identity does not yet fold the closure sidecar (`closure_hash`), and graph-driven cache eviction awaits 30 days of accumulated run history.

Verification

Every number in this paper is reproducible on the tagged tree: `mir-corpus/gates/run-gates.sh` (the eleven gate values), `fluxc tdg plan` after an edit (cone sizes), `time fluxc self` twice (the warm invariant), `chronos-v035/EVIDENCE/ladder.log` (the storage verdict), `flux-rev snapshot .` (the release identity).